

The CTX-pro family: finer-grained coverage keeps the seed that goes deeper

Backdrop. *CTX-pro* collects the roadblocks where a fuzzer with *context-sensitive coverage* (`naive_ctx`) gets through while the same byte-mutation fuzzer with a plain edge map (`naive`) stays stuck. Both fuzzers mutate bytes the same blind way — the *only* difference is what counts as “new coverage.” Plain `naive` has one slot per control edge: the first input to cross edge E marks it done, and any later seed that crosses E again — from a deeper loop iteration, a deeper recursion level, or a *different caller chain* — earns no credit and is thrown away. `naive_ctx`’s `CtxHook` folds the *caller chain* into the map index, so each *(call-context, edge)* pair is its own slot. Re-entering the same edge one loop deeper, or reaching the same function through a new caller, now registers as fresh coverage — so the seed is *kept*. That retention manufactures a *depth gradient*: the corpus accumulates partial, mid-progress seeds, and a rare deep continuation becomes reachable from a retained mid-parse seed instead of having to appear all at once. Unlike I2S/VP there is no “key beside the lock” here — there is no value to paste; the advantage is purely *which seeds survive*. Counts are validated roadblocks (n). Of the 53 CTX-pro memberships, **14 validate** as the one discriminable mechanism below; the remaining 39 are honestly *inconclusive* (see “The other two faces”).

1. Iteration / path-depth reach

`ctx_iteration_path_depth` ($n = 14$)

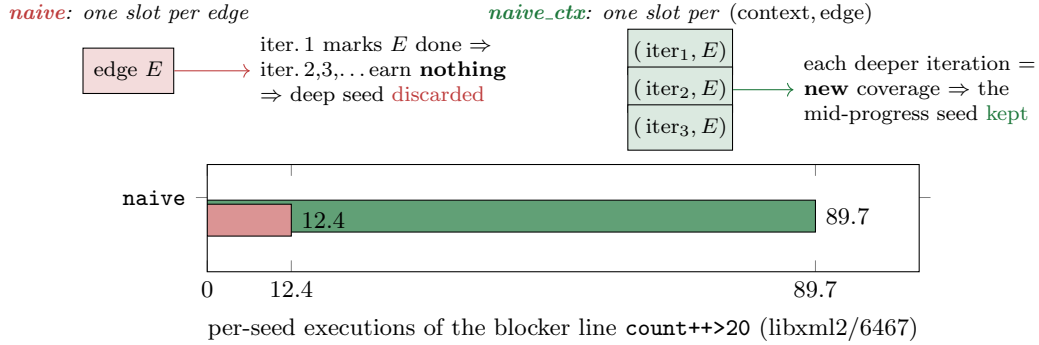
The blocker sits one step *deeper* along a path the loser corpus never retains seeds for: an extra loop iteration, a deeper recursion level, the same function reached through a new call context. `naive`’s saturated edge map discards the mid-progress seeds that would get there; `naive_ctx`’s context-split map keeps them, so its corpus drives the blocker line to a strictly higher per-seed execution count (or actually takes the blocked side). This is the *only* face of the `ctx` advantage that shows up per-seed, and the only CTX-pro mechanism real campaign data can discriminate.

```
// libxml2 parser.c : xmlParseCharRef() -- decimal char reference &#
    dddd...;
// the digit-scan loop; the gate is one extra iteration past a depth
    threshold
while (RAW != ';' ) {                                // RAW walks the &#... digits
    val = val * 10 + (CUR - '0');
    if (count++ > 20) {                                // <-- blocker (libxml2/6467,
        L2353):
        xmlFatalErr(...); return 0;                    // only seeds with >20
        digits reach it
    }
    NEXTL(...);
}
```

How we know it is really the depth mechanism. If the winner corpus is winning by reaching the line *deeper*, then running each arm’s saved seeds through the instrumented binary should show the `naive_ctx` seeds executing the blocker line far more times per seed (and taking the blocked side)

than **naive**’s. We run the seeds and read the line’s execution count straight from the `llvm-cov` dump; the rule asks for at least a $2\times$ per-seed lift *and* that the winner is the deeper one.

Tool: <code>depth_reach</code>
Metric: $\text{depth_lift} = \frac{\text{winner per-seed blocker-line execs}}{\max(\text{loser per-seed blocker-line execs}, 1)}$; winner_deeper = winner reaches the line strictly deeper
Verify rule (the arbiter fires this, no LLM): <code>depth_lift >= 2.0 AND winner_deeper == True</code>
Real example: <code>libxml2/6467 (xmlParseCharRef, count++>20)</code> — the <code>naive_ctx</code> corpus runs the gate line 89.7 times per seed vs <code>naive</code> ’s 12.4, so <code>depth_lift</code> = 7.26 and the blocked side is taken $7.0\times$ more (<code>side_lift</code> = 7.02): only the context-retained corpus carries the >20-digit references that trip the loop past threshold.



Same blind byte mutation on both arms; the context-split map keeps the seeds that re-enter the digit loop, so `naive_ctx` drives the gate line $7.26\times$ deeper (`depth_lift`) and trips the >20-iteration side the loser never assembles.

What the rule rules out. A value/byte-solving roadblock mis-bucketed into this `naive_ctx>naive` shape would reach the blocker line at the *same* depth and differ only in operand bytes — `depth_lift`≈ 1, rule fails. A branch where `naive` is actually the deeper arm (e.g. `harfbuzz/5237`, `depth_lift`= 0.14) fails on `winner_deeper`. So the rule is both falsifiable and discriminating: it fires only when the winner’s edge is genuinely the deeper-reaching corpus.

The other two faces (why only 14 of 53 validate)

CTX-pro is a *non-local* feedback mechanism, and most of its surface does not leave a per-seed fingerprint. We tested two further faces honestly; neither rescued the residual, and we leave them on the record rather than force a label.

Face 2 — corpus-scale inflation (refuted on the measured arms). The textbook story for context-sensitive coverage is that it admits *many more* seeds — the same edge through a new caller chain counts as new coverage — so `naive_ctx` should hoard a strictly larger, more context-diverse saved corpus than `naive`, whether or not any single seed reaches the blocker deeper. That is a corpus-scale signature `depth_reach` is blind to, so we built a separate arm-level test.

Tool: <code>corpus_size_ratio</code>
Metric: $\text{corpus_count_ratio} = \frac{\text{median \# saved seeds, naive_ctx}}{\text{median \# saved seeds, naive}}$ across trials (> 1 = ctx hoards more)
Verify rule (the arbiter fires this, no LLM): <code>corpus_count_ratio >= 1.5</code>
Real example: Measured on this shape’s targets: <code>harfbuzz</code> = 0.924, <code>sqlite3</code> = 0.956 (composition-entropy ratios ≈ 1.01) — both <i>below</i> 1.0, so the corpus-inflation mechanism is refuted on the measured arms. The rule is kept as a genuine falsifiable test (it <i>can</i> fire ≥ 1.5 on a target where ctx truly inflates), but here it correctly does not fire.

Face 3 — the value route is structurally unavailable (*decidable:false*). The 39 remaining branches fail both kept rules: their per-seed depth does not separate (`depth_lift < 2.0`) and their arm does not hoard a bigger corpus (`corpus_count_ratio < 1.5`). Several *do* carry a literal operand (`libpng color_type==3`; `libxml2 overlong-encoding`; `harfbuzz codepoint switch arms`), which is why a byte/value tool looks tempting — but the resolving arm is `naive_ctx`, which has *neither* `value_profile` *nor* `cmplog`. There is no I2S operand-substitution contrast for `operand_enrichment` to read and no Hamming gradient for `value_distance_reached` to climb; the cards say plainly the bytes are *not* the blocker (“no extra byte requirement beyond naive’s”) — retention/scheduling is. Applying a value tool here would manufacture a comparison the mechanism cannot produce (a G2 violation: the winning arm has no value-solver). This is a *mechanism-level* rejection valid for every residual branch on both servers, so the subgroup converges honestly to `decidable:false` rather than to a forced label.

At a glance

Face	<i>n</i>	Tool / metric	Outcome / one-line reason
iteration / path-depth reach	14	<code>depth_reach</code> / <code>depth_lift</code> , <code>winner_deeper</code>	validated: context-split map keeps the seed that re-enters the loop / reaches the line deeper
corpus-scale inflation	0	<code>corpus_size_ratio</code> / <code>corpus_count_ratio</code>	<i>refuted</i> on measured arms (0.92, 0.96 < 1.5); kept as a falsifiable test
value route (literal operand)	0	<code>operand_enrichment</code> / <code>value_distance_reached</code>	decidable:false: <code>naive_ctx</code> has no I2S/VP arm to read (G2)
CTX-pro validated	14		(of 53 memberships; 39 honestly inconclusive)

Counts and example metrics regenerated from `bench/dataset.jsonl` / `bench/roadblock.facts.csv`; verify rules are the deterministic arbiter rules (no LLM in the decision). The refuted and `decidable:false` faces are reported deliberately — an honest **inconclusive** is the correct outcome for a non-local mechanism with no cheap discriminating signal, not a gap to paper over.